

Einführung: Trigger in PL/SQL

Was ist ein Trigger

Eigenschaften von Triggern:

- Ein Trigger ist ein benannter PL/SQL-Block, der aufgerufen wird, wenn ein festgelegtes Ereignis in der Datenbank auftritt. Er kann nicht direkt aufgerufen werden wie eine gespeicherte Prozedur.
- Ein Trigger hat keine Parameter.
- Der Benutzer kann nicht feststellen, ob ein oder mehrere Trigger ausgelöst wurden.
- Es gibt Trigger, die durch DML-Kommandos ausgelöst werden.
- Ein Trigger kann auf Änderungen bestimmter Spalten eingeschränkt werden.
- Es gibt Trigger, die durch DDL-Kommands ausgelöst werden.
- Es gibt Trigger, die durch Datenbankereignisse wie Shutdown oder Benutzer-Login ausgelöst werden.

Anwendungsgebiete von Triggern

- Ergänzung von deklarativer referentieller Integrität
- Erzwingen von Geschäftsregeln
- Überwachung von Datenänderungen
- Überwachung von Änderungen der Datenbankstruktur

Foreign-Key-Constraints und Check-Constraints sind das Mittel erster Wahl zum Erzwingen von Geschäftsregeln. Nur wenn das nicht möglich ist, sollen Trigger eingesetzt werden. Da die Ausführung von Triggern nicht erkennbar ist, kann deren unüberlegter und übermäßiger Einsatz zu unerklärlichen Phänomenen und Performanceproblemen in der Datenbank führen.

Erforderliche System- und Objektberechtigungen

Um einen Trigger für eine Tabelle anlegen zu können, oder einen Trigger zu de- bzw. reaktivieren, benötigen Sie eine der folgenden Berechtigungen:

- Sie sind Besitzer der Tabelle.
- Sie haben die ALTER-Berechtigung für die Tabelle.
- Sie haben die Systemberechtigung ALTER ANY TABLE.

Weiters benötigen Sie eine dieser Berechtigungen:

- Die Systemberechtigung CREATE TRIGGER.
- Zum Anlegen von Triggern in einem Fremdschema CREATE ANY TRIGGER.

Zum Ändern von Triggern:

- Sie sind Eigentümer des Triggers.
- Sie haben die Systemberechtigung ALTER ANY TRIGGER.

Für Trigger auf Datenbankebene benötigen Sie die Systemberechtigung ADMINISTER DATABASE TRIGGER. Führt der Trigger Änderungen an anderen Tabellen durch, benötigen Sie die entsprechenden Berechtigungen für diese Tabellen. Diese Berechtigungen müssen dem Besitzer des Triggers direkt, also nicht über Rollen, erteilt worden sein.

DML-Trigger

- BEFORE-Trigger werden vor der Datenänderung aufgerufen.
- AFTER-Trigger werden nach der Datenänderung aufgerufen.
- Datenänderungen können nur in einem BEFORE-Trigger durchgeführt werden.
- Auf alte und geänderte Werte kann über die Strukturen OLD und NEW zugegriffen werden.

Bei Oracle dürfen Trigger keine Transaktionsbefehle wie SAVEPOINT, ROLLBACK oder COMMIT verwenden. Der Trigger lässt sich zwar kompilieren, bricht zur Laufzeit aber mit einer Fehlermeldung ab.

Syntax CREATE TRIGGER

```
CREATE [OR REPLACE] TRIGGER trigger_name Erste Zeile
{BEFORE | AFTER}
{INSERT | UPDATE | UPDATE OF column1 [, column2 [, column(n+1)]] | DELETE}
ON table_name
[FOR EACH ROW]
[WHEN (logical_expression)]
[DECLARE]
[PRAGMA AUTONOMOUS_TRANSACTION;]
declaration_statements;
BEGIN
execution_statements;
END [trigger_name];
/
```

Nameskonventionen für DML-Trigger

- Triggernamen dürfen maximal 30 Zeichen lang sein.
- Sie sollen den Tabellennamen enthalten.
- Ob es ein AFTER- oder BEFORE-Trigger ist.
- Auf welche DML-Kommandos der Trigger reagiert.
- Ob es ein Row- oder Statement-Level Trigger ist.

Beispiele:

- parts_aft_upd_del_row
- suppliers_bef_ins_sta

Trigger auf Datensatzebene (Row-Level Trigger)

Trigger auf Datensatzebene (Row-Level Triggers) sind die häufigste Triggerart und werden einmal für jeden durch ein DML-Kommando betroffenen Datensatz aufgerufen. Sie werden mit der **FOR EACH ROW**-Klausel angelegt.

Beispiel:

```
create or replace trigger parts_aft_ins_upd_row
after insert or update
on parts
for each row
begin
  if inserting then
    dbms_output.put_line('Yikes! Insert on table parts detected.');
```

```
  else
    dbms_output.put_line('Hey, update on table parts detected.');
```

```
  end if;
end;
/
show errors
```

Innerhalb eines Triggers kann mit den Prädikaten **INSERTING**, **UPDATING** bzw. **DELETING** geprüft werden, welches DML-Kommando den Aufruf des Triggers verursacht hat.

Bearbeiten Sie Übung 1.

UPDATE auf bestimmte Felder

Ein DML-Trigger kann so angelegt werden, dass er nur auf Änderungen bestimmter Felder mit **UPDATE** reagiert.

Beispiel:

```
create or replace trigger parts_aft_upd_row
after update of PartColor
on parts
for each row
begin
  dbms_output.put_line('Attention, part has changed color!');
```

```
end;
/
show errors
```

Bearbeiten Sie Übung 2.

Alte und neue Werte

Innerhalb eines Triggers auf Datensatzebene stehen die Pseudo-Datensätze **NEW** und **OLD** zur Verfügung. Sie enthalten die Werte nach bzw. vor der Änderung.

Beispiel:

```
create or replace trigger parts_aft_upd_color
after update of PartColor
on parts
for each row
begin
    dbms_output.put_line('Attention, part has changed color from ' ||
        trim(:old.PartColor) || ' to ' ||
        trim(:new.PartColor) || '!');
end;
/
show errors
```

Erscheint im SQL Developer bei Verwendung von **:** ein Eingabefenster, kann das durch Ausführung von **set define off** unterbunden werden.

Bearbeiten Sie Übung 3. Bearbeiten Sie Übung 4.

WHEN-Klausel

Mit der **WHEN**-Klausel können zusätzliche Bedingungen angegeben werden, die mitbestimmen, ob der Trigger ausgelöst wird oder nicht.

Beispiel:

```
create or replace trigger parts_aft_upd_when
after update of PartColor
on parts
for each row
when (:new.PartColor = 'pink')
begin
    dbms_output.put_line('I am a trigger and I hate PINK!');
end;
/
show errors
```

Wenn Sie in der **WHEN**-Klausel die Pseudo-Datensätze **OLD** bzw. **NEW** verwenden, dürfen Sie keinen Doppelpunkt voranstellen. Das obenstehende Beispiel ist syntaktisch nicht korrekt!

Bearbeiten Sie Übung 5.

Beachten Sie bei der **WHEN**-Klausel:

- Die Bedingung muss mit Klammern umgeben sein.
- Kein **:** vor **OLD** und **NEW**.
- SQL-Funktionen in der **WHEN**-Klausel sind erlaubt, nicht jedoch benutzerdefinierte Funktionen oder in Packages definierte Funktionen.
- **WHEN**-Klauseln sind nur in Row-Level Triggern erlaubt.

Trigger aktivieren, deaktivieren und verwerfen

Ein Trigger ist automatisch aktiviert, wenn er angelegt wird. Aktive Trigger können beim Laden großer Datenmengen sehr große Performanceeinbußen zur Folge haben. Manchmal ist es daher erforderlich, einen Trigger zu deaktivieren, beispielsweise beim Laden großer Datenmengen. Die Änderungen, die der Trigger bewirkt, müssen anschließend händisch nachgezogen werden. Anschließend wird der Trigger wieder aktiviert.

Einzelnen Trigger deaktivieren:

```
alter trigger parts_aft_upd_when disable;
```

Alle Trigger einer Tabelle deaktivieren:

```
alter table parts disable all triggers;
```

Einzelnen Trigger bzw. alle Trigger einer Tabelle aktivieren:

```
alter trigger parts_aft_upd_when enable;  
alter table parts enable all triggers;
```

Trigger verwerfen, d.h. löschen:

```
drop trigger parts_aft_upd_when;
```

Ungültig gewordene Trigger können mit **ALTER TRIGGER COMPILE** neu compiliert werden.

Beispiel:

```
alter trigger parts_aft_upd_color compile;
```

Bearbeiten Sie Übung 6.

BEFORE-Trigger

In einem BEFORE-Trigger können über den Pseudodatensatz **NEW** auch Wertänderungen vorgenommen werden, indem den Feldern einfach Werte zugewiesen werden.

Bearbeiten Sie Übung 7.

Um Datenänderungen rückgängig zu machen bzw. zu verhindern, muss ein Anwendungsfehler ausgelöst werden.

Beispiel:

```
raise_application_error(-20001, 'Read-only table, no changes allowed.');
```

Bearbeiten Sie Übung 8.

Trigger auf Befehlsebene (Statement-Level Trigger)

Trigger auf Befehlsebene (Statement-Level Trigger) werden für jedes DML-Kommando nur einmal ausgeführt. Wenn also ein DELETE-Kommando 20 Datensätze löscht, wird der Trigger ein einziges Mal ausgeführt. Diese Triggerart wird meist verwendet, um unerwünschte Datenänderungen zu

unterbinden. Sie sind die voreingestellte Triggerart. Sie werden manchmal auch als Trigger auf Tabellenebene (Table-Level Trigger) bezeichnet.

Mutating Table Errors

Ein **ORA-04091** Mutating Table Error tritt auf, wenn in einem Row-Level Trigger versucht wird, die Tabelle zu lesen oder zu ändern, die den Trigger ausgelöst hat. Richtlinien dazu:

- Trigger auf Datensatzebene dürfen die Tabelle, die den Trigger ausgelöst hat, weder lesen noch ändern. Diese Einschränkung gilt nicht für Trigger auf Befehlsebene.
- Verwendet der Trigger mit **PRAGMA AUTONOMOUS TRANSACTION** im Deklarationsteil eine autonome Transaktion mit **COMMIT** im Ausführungsteil, darf die auslösende Tabelle gelesen, aber nach wie vor nicht geändert werden.

Kombinierte Trigger (Compound Trigger)

Ein kombinierter Trigger (Compound Trigger) kann mehrere Triggertypen zusammenfassen (möglich seit Oracle 11g).

INSTEAD OF-Trigger

- Der Code des Triggers wird statt der Änderungsoperation ausgeführt.
- Diese Triggerart kann nur für Views angelegt werden, nicht für Tabellen.
- Anwendungsbeispiel: Änderung von Daten einer View, die aus mehreren Tabellen besteht.

DDL-Trigger

- Werden auf Operationen wie **CREATE TABLE** definiert.
- Auch als BEFORE DDL-Trigger.
- Zur Verhinderung von DDL-Operationen und zur Sicherheitsüberwachung von DDL-Operationen.

Trigger auf Datenbankebene

- Für Datenbankereignisse wie beispielsweise für Login, Logoff, Shutdown, Startup etc.
- Zur Sicherheitsüberwachung und für die Datenbankwartung.
- Voraussetzung für Virtual Private Databases (VPD).

Übungen

Übung 1

1. Laden Sie die Beispieltabellen Catalog in Ihre Datenbank.
2. Legen Sie den Beispieltrigger an.
3. Zeigen Sie anhand von zwei DML-Kommandos, dass der Trigger auf INSERT und UPDATE reagiert.
4. Machen Sie die zwei DML-Kommandos wieder rückgängig.

Übung 2

1. Legen Sie den Beispieltrigger an.
2. Zeigen Sie anhand von zwei DML-Kommandos, dass der Trigger nur auf ein UPDATE der Spalte PartColor reagiert.
3. Machen Sie die zwei DML-Kommandos wieder rückgängig.

Übung 3

1. Legen Sie den Beispieltrigger an.
2. Zeigen Sie anhand eines DML-Kommandos, dass der Trigger bei einem UPDATE der Spalte PartColor die Farbe vor und nach der Änderung ausgibt.
3. Machen Sie das DML-Kommando wieder rückgängig.

Übung 4

1. Schreiben Sie einen After-Trigger auf der Tabelle Suppliers, der auf alle drei DML-Kommandos anspricht.
2. Geben Sie aus, welche Art von Ereignis den Trigger ausgelöst hat.
3. Geben Sie Werte der Felder vor und nach der Änderung aus.
4. Führen Sie je eine Beispieländerung für die drei DML-Arten aus.
5. Machen Sie Änderungen an der Tabelle rückgängig.

Beispielausgabe für UPDATE:

```
Triggered by UPDATE
SupplierID OLD: L6 NEW: L6
SupplierName OLD: Miller NEW: Miller
SupplierCity OLD: York NEW: Dublin
SupplierDiscount OLD: 10 NEW: 10
```

Übung 5

1. Legen Sie den Beispieltrigger an.
2. Der Beispielcode hat einen Syntaxfehler. Beheben Sie diesen Fehler.
3. Zeigen Sie anhand von zwei UPDATE-Kommandos, dass der Trigger bei einem UPDATE der Spalte PartColor nur dann reagiert, wenn die Farbe den Wert pink erhält.
4. Machen Sie das DML-Kommando wieder rückgängig.

Übung 6

1. Deaktivieren Sie den Trigger parts_aft_upd_when. Zeigen Sie, dass der Trigger nicht mehr ausgelöst wird.
2. Aktivieren Sie den Trigger wieder. Zeigen Sie, dass er wieder ausgelöst wird.
3. Deaktivieren Sie alle Trigger auf der Tabelle Parts. Zeigen Sie, dass die Trigger nicht mehr ausgelöst werden.
4. Reaktivieren Sie die Trigger und zeigen Sie, dass das der Fall ist.
5. Löschen Sie den Trigger parts_aft_upd_when. Zeigen Sie, dass der Trigger weg ist.

Übung 7

1. Erstellen Sie einen BEFORE-Trigger, der Farbänderungen auf pink abfängt. Achten Sie auf die Namenskonventionen.
2. Die Farbänderung soll auf lila abgeändert werden, nachdem eine entsprechende Meldung ausgegeben wurde.
3. Zeigen Sie anhand von zwei UPDATE-Kommandos, dass der Trigger bei einem UPDATE der Spalte PartColor nur dann reagiert, wenn die Farbe den Wert pink erhält.
4. Fragen Sie den geänderten Datensatz ab.
5. Machen Sie das DML-Kommando wieder rückgängig.

Übung 8

1. Erstellen Sie einen BEFORE-Trigger, der Farbänderungen auf pink abfängt. Achten Sie auf die Namenskonventionen.
2. Die Datenänderung soll verhindert werden mit der Fehlermeldung:

```
There is already enough pink in the world. Choose another color. Operation blocked..
```


3. Zeigen Sie anhand von zwei UPDATE-Kommandos, dass der Trigger bei einem UPDATE der Spalte PartColor nur dann reagiert, wenn die Farbe den Wert pink erhält.
4. Zeigen Sie, dass durch das UPDATE keine Änderung stattgefunden hat.